

Introduction to JAX

Introduction to JAX: A Practical Guide for Embedded Machine Learning

Introduction

JAX is a Python library developed by Google for high performance numerical computation and machine learning. It almost looks like Numpy and is sometimes referred to as “NumPy on steroids”. Its core pillars are:

- Automatic Differentiation (AD): Computing gradients of arbitrary functions
- Just-In-Time (JIT) Compilation: Compile Python code to GPU/TPU using XLA
- Automatic Vectorization (vmap): Transform scalar functions to work on batches
- Functional Programming: Pure functions enable composability
- Parallelization Across Devices (pmap): Distribute computations across all GPUs at once

Note: JAX strictly relies on functional programming. Therefore the code must be “pure”, functions cannot have side effects, and arrays are immutable but instead, you have to create a new array with the update.

Why JAX for Machine Learning? * Performance: JIT compilation gives 10-100x speedups * Research-Friendly: Easy to experiment with new algorithms * Embedded ML: Lightweight, efficient inference on edge devices * Composability: Stack transformations (grad-vmap-pjit) * Control: Fine-grained control over computation graph

Installation

```
[ ]: # Basic installation
pip install jax jaxlib
# GPU support ( CUDA 12)
pip install -- upgrade " jax [ cuda12_cudnn ] "
# TPU support
pip install -- upgrade " jax [ tpu ] " -f https://storage.googleapis.com/jax-releases/libtpu_releases.html
# Verify installation
python -c " import jax ; print ( jax . devices () ) "
```

1. Just-In-Time Compilation

1. Just-In-Time Compilation (JIT)

JAX allows us to transform Python functions by reducing each function into a sequence of primitive operations, each representing one fundamental unit of computation. Primitives are units of computation; most functions in `jax.lax` represent a primitive. JAX utilizes an optimization engine called XLA (Accelerated Linear Algebra) to compile your Python functions into well optimized machine code. By simply wrapping a function in `jit`, JAX merges operations together, minimizes memory transfers, and runs them incredibly fast.

JIT compiling a function

JAX enables operations to execute on CPU/GPU/TPU using the same code. Below is an example of computing a Scaled Exponential Linear Unit (SELU), an operation commonly used in deep learning:

```
[3]: import jax
import jax . numpy as jnp
def selu ( x , alpha =1.67 , lambda_ =1.05 ) :
    return lambda_ * jnp . where ( x > 0 , x , alpha * jnp . exp ( x ) - alpha )
x = jnp . arange (1000000)
%timeit selu ( x ) . block_until_ready ()
```

9.15 ms $\pm$ 1.3 ms per loop (mean $\pm$ std. dev. of 7 runs, 1 loop each)

The code above is sending one operation at a time to the accelerator. This limits the ability of the XLA compiler to optimize our functions. Naturally, what we want to do is give the XLA compiler as much code as possible, so it can fully optimize it. For this purpose, JAX provides the `jax.jit()` transformation, which will JIT compile a JAX-compatible function. Below is an example of JIT in operation.

```
[4]: selu_jit = jax . jit ( selu )
# Pre - compile the function before timing ...
selu_jit ( x ) . block_until_ready ()
%timeit selu_jit ( x ) . block_until_ready ()
```

1.41 ms $\pm$ 316 μ s per loop (mean $\pm$ std. dev. of 7 runs, 1000 loops each)

`selu_jit` was defined as the compiled version of `selu` and called `selu_jit` once on `x`. This is where JAX does its tracing, it needs to have some inputs to wrap in tracers, after all. The `jaxpr` is then compiled using XLA into very efficient code optimized for your GPU or TPU. Finally, the compiled code is executed to satisfy the call. We can't `jit` everything, common examples are;

- Cannot JIT functions that use Python conditional statements or loops.
- Dynamic Array Shapes
- I/O operations

2. Automatic Vectorization (vmap)

2. Automatic Vectorization (vmap)

Consider the following simple code that computes the convolution of two one- dimensional vectors:

```
[1]: import jax
import jax.numpy as jnp

x = jnp.arange(5)
w = jnp.array([2., 3., 4.])

def convolve(x, w):
    output = []
    for i in range(1, len(x)-1):
        output.append(jnp.dot(x[i-1:i+2], w))
    return jnp.array(output)

convolve(x, w)
```

```
[1]: Array([11., 20., 29.], dtype=float32)
```

Suppose we would like to apply this function to a batch of weights w to a batch of vectors x .

```
[3]: xs = jnp.stack([x, x])
ws = jnp.stack([w, w])
```

The most naive option would be to simply loop over the batch in Python:

```
[4]: def manually_batched_convolve(xs, ws):
    output = []
    for i in range(xs.shape[0]):
        output.append(convolve(xs[i], ws[i]))
    return jnp.stack(output)

manually_batched_convolve(xs, ws)
```

```
[4]: Array([[11., 20., 29.],
           [11., 20., 29.]], dtype=float32)
```

The result is correct but not very efficient. For example, we could manually rewrite `convolve()`

to support vectorized computation across the batch dimension in order to batch the computation efficiently, as follows:

```
[5]: def manually_vectorized_convolve(xs, ws):  
    output = []  
    for i in range(1, xs.shape[-1] - 1):  
        output.append(jnp.sum(xs[:, i-1:i+2] * ws, axis=1))  
    return jnp.stack(output, axis=1)  
  
manually_vectorized_convolve(xs, ws)
```

```
[5]: Array([[11., 20., 29.],  
           [11., 20., 29.]], dtype=float32)
```

This re-implementation can be messy and error-prone as the complexity of a function increases; that is where JAX comes in with *Automatic Vectorization*.

vmap

In JAX, the `jax.vmap()` transformation is designed to generate such a vectorized implementation of a function automatically:

```
[6]: auto_batch_convolve = jax.vmap(convolve)  
  
auto_batch_convolve(xs, ws)
```

```
[6]: Array([[11., 20., 29.],  
           [11., 20., 29.]], dtype=float32)
```

It does this by tracing the function similarly to `jax.jit()`, and automatically adding batch axes at the beginning of each input. If the batch dimension is not the first, you may use the `inaxes` and `outaxes` arguments to specify the location of the batch dimension in inputs and outputs. These may be an integer if the batch axis is the same for all inputs and outputs, or lists, otherwise.

```
[7]: auto_batch_convolve_v2 = jax.vmap(convolve, in_axes=1, out_axes=1)  
  
xst = jnp.transpose(xs)  
wst = jnp.transpose(ws)  
  
auto_batch_convolve_v2(xst, wst)
```

```
[7]: Array([[11., 11.],  
           [20., 20.],  
           [29., 29.]], dtype=float32)
```

Combining Transformations

As with all JAX transformations, `jax.jit()` and `jax.vmap()` are designed to be com-posable, which means you can wrap a vmapped function with `jit`, or a jitted function with `vmap`, and everything will work correctly:

```
[8]: batch_convolve_v3 = jax.vmap(convolve, in_axes=[0, None])  
  
batch_convolve_v3(xs, w)
```

```
[8]: Array([[11., 20., 29.],  
           [11., 20., 29.]], dtype=float32)
```

3. Automatic differentiation(autodiff)

[]:

3. Automatic differentiation(autodiff)

JAX has a good general autodiff system. Computing gradients is a critical part of modern machine learning methods.

Taking gradients with `jax.grad`

In JAX, you can differentiate a scalar-valued function with the `jax.grad()` transformation:

```
[ ]: import jax
import jax . numpy as jnp
from jax import grad
grad_tanh = grad ( jnp . tanh )
print ( grad_tanh (2.0) )
```

`jax.grad()` takes a function and returns a function. If you have a Python function `f` that evaluates the mathematical function, then `jax.grad(f)` is a Python function that evaluates the mathematical function. That means `grad(f)(x)` represents the value. Since `jax.grad()` operates on functions, you can apply it to its own output to differentiate as many times as you like:

```
[ ]: print(grad ( grad ( jnp . tanh ) ) (2.0) )
print(grad ( grad ( grad ( jnp . tanh ) ) ) (2.0) )
```

JAX's autodiff makes it easy to compute higher-order derivatives, because the functions that compute derivatives are themselves differentiable. Thus, higher-order derivatives are as easy as stacking transformations. See the example below; The derivative of the equation below can be computed as shown in the code snippet below;

$$f(x) = x^3 + 2x^2 - 3x + 1$$

```
[ ]: f = lambda x : x **3 + 2* x **2 - 3* x + 1
dfdx = jax . grad ( f )
```

The higher order derivatives of `f` are;

$$f'(x) = 3x^2 + 4x - 3$$

$$f''(x) = 6x + 4$$

$$f'''(x) = 6$$

$$f^{iv}(x) = 0$$

In JAX, computing any of these is as easy as chaining the `jax.grad()` function:

```
[ ]: d2fdx = jax.grad(dfdx)
      d3fdx = jax.grad(d2fdx)
      d4fdx = jax.grad(d3fdx)

      # Evaluating for x = 1
      print ( dfdx (1.) )
      print ( d2fdx (1.) )
      print ( d3fdx (1.) )
      print ( d4fdx (1.) )
```

Computing gradients in a linear logistic regression

Consider the logistic regression model below;

```
[ ]: key = jax.random.key(0)

def sigmoid(x):
    return 0.5 * (jnp.tanh(x / 2) + 1)

# Outputs probability of a label being true.
def predict(W, b, inputs):
    return sigmoid(jnp.dot(inputs, W) + b)

# Build a toy dataset.
inputs = jnp.array([[0.52, 1.12, 0.77],
                    [0.88, -1.08, 0.15],
                    [0.52, 0.06, -1.30],
                    [0.74, -2.49, 1.39]])
targets = jnp.array([True, True, False, True])

# Training loss is the negative log-likelihood of the training examples.
def loss(W, b):
    preds = predict(W, b, inputs)
    label_probs = preds * targets + (1 - preds) * (1 - targets)
    return -jnp.sum(jnp.log(label_probs))

# Initialize random model coefficients
key, W_key, b_key = jax.random.split(key, 3)
W = jax.random.normal(W_key, (3,))
```

```
b = jax.random.normal(b_key, ())
```

Use the `jax.grad()` function with its `argnums` argument to differentiate a function with respect to positional arguments.

```
[ ]: # Differentiate `loss` with respect to the first positional argument:
W_grad = grad(loss, argnums=0)(W, b)
print(f'{W_grad=}')

# Since argnums=0 is the default, this does the same thing:
W_grad = grad(loss)(W, b)
print(f'{W_grad=}')

# But you can choose different values too, and drop the keyword:
b_grad = grad(loss, 1)(W, b)
print(f'{b_grad=}')

# Including tuple values
W_grad, b_grad = grad(loss, (0, 1))(W, b)
print(f'{W_grad=}')
print(f'{b_grad=}')

```

Differentiating with respect to nested lists, tuples, and dicts

`jax.grad()` can also work with dictionaries for parameters. It returns the output with respect to each parameter (key) as a separate output dictionary item.

```
[ ]: def loss2(params_dict):
    preds = predict(params_dict['W'], params_dict['b'], inputs)
    label_probs = preds * targets + (1 - preds) * (1 - targets)
    return -jnp.sum(jnp.log(label_probs))

print(grad(loss2)({'W': W, 'b': b}))

```

Evaluating a function and its gradient

`jax.value_and_grad()` will compute a function's value as well as its gradient's value in one pass.

```
[ ]: loss_value, Wb_grad = jax.value_and_grad(loss, (0, 1))(W, b)
print('loss value', loss_value)
print('loss value', loss(W, b))

```

4. Pytrees

Pytrees

A pytree is a container-like structure built out of container-like Python objects — “leaf” pytrees and/or more pytrees. A pytree can include lists, tuples, and dicts. A leaf is anything that’s not a pytree, such as an array, but a single leaf is also a pytree.

In the context of machine learning (ML), a pytree can contain:

- Model parameters
- Dataset entries
- Reinforcement learning agent observations

When working with datasets, you can often come across pytrees (such as lists of lists of dicts).

Below is an example of a simple pytree. In JAX, you can use `jax.tree.leaves()`, to extract the flattened leaves from the trees, as demonstrated here:

```
[1]: import jax
import jax.numpy as jnp

example_trees = [
    [1, 'a', object()],
    (1, (2, 3), ()),
    [1, {'k1': 2, 'k2': (3, 4)}, 5],
    {'a': 2, 'b': (2, 3)},
    jnp.array([1, 2, 3]),
]

# Print how many leaves the pytrees have.
for pytree in example_trees:
    # This `jax.tree.leaves()` method extracts the flattened leaves from the
    # pytrees.
    leaves = jax.tree.leaves(pytree)
    print(f"{repr(pytree):<45} has {len(leaves)} leaves: {leaves}")
```

```
[1, 'a', <object object at 0x7e0d48391d70>]    has 3 leaves: [1, 'a', <object
object at 0x7e0d48391d70>]
(1, (2, 3), ())                                has 3 leaves: [1, 2, 3]
[1, {'k1': 2, 'k2': (3, 4)}, 5]                 has 5 leaves: [1, 2, 3, 4, 5]
{'a': 2, 'b': (2, 3)}                          has 3 leaves: [2, 2, 3]
```

```
Array([1, 2, 3], dtype=int32)          has 1 leaves: [Array([1, 2, 3],
dtype=int32)]
```

Common pytree functions

JAX provides a number of utilities to operate over pytrees. These can be found in the `jax.tree_util` subpackage; for convenience many of these have aliases in the `jax.tree` module.

The most common function is the `jax.tree.map`. It works analogously to Python's native `map`, but transparently operates over entire pytrees. Below is an example.

```
[2]: list_of_lists = [
      [1, 2, 3],
      [1, 2],
      [1, 2, 3, 4]
    ]

    jax.tree.map(lambda x: x*2, list_of_lists)
```

```
[2]: [[2, 4, 6], [2, 4], [2, 4, 6, 8]]
```

`jax.tree.map()` also allows mapping a N-ary function over multiple arguments.

```
[3]: another_list_of_lists = list_of_lists
    jax.tree.map(lambda x, y: x+y, list_of_lists, another_list_of_lists)
```

```
[3]: [[2, 4, 6], [2, 4], [2, 4, 6, 8]]
```

Illustration of `jax.tree.map` with ML model parameters

This example demonstrates how pytree operations can be useful when training a simple multi-layer perceptron (MLP).

We start by defining the initial model parameters:

```
[4]: import numpy as np

    def init_mlp_params(layer_widths):
        params = []
        for n_in, n_out in zip(layer_widths[:-1], layer_widths[1:]):
            params.append(
                dict(weights=np.random.normal(size=(n_in, n_out)) * np.sqrt(2/n_in),
                    biases=np.ones(shape=(n_out,)))
            )
        return params

    params = init_mlp_params([1, 128, 128, 1])
```

Use `jax.tree.map()` to check the shapes of the initial parameters:

```
[5]: jax.tree.map(lambda x: x.shape, params)
```

```
[5]: [{'biases': (128,), 'weights': (1, 128)},  
      {'biases': (128,), 'weights': (128, 128)},  
      {'biases': (1,), 'weights': (128, 1)}]
```

Next, define the functions for training the MLP model:

```
[6]: # Define the forward pass.  
def forward(params, x):  
    *hidden, last = params  
    for layer in hidden:  
        x = jax.nn.relu(x @ layer['weights'] + layer['biases'])  
    return x @ last['weights'] + last['biases']  
  
# Define the loss function.  
def loss_fn(params, x, y):  
    return jnp.mean((forward(params, x) - y) ** 2)  
  
# Set the learning rate.  
LEARNING_RATE = 0.0001  
  
# Using the stochastic gradient descent, define the parameter update function.  
# Apply `@jax.jit` for JIT compilation (speed).  
@jax.jit  
def update(params, x, y):  
    # Calculate the gradients with `jax.grad`.  
    grads = jax.grad(loss_fn)(params, x, y)  
    # Note that `grads` is a pytree with the same structure as `params`.  
    # `jax.grad` is one of many JAX functions that has  
    # built-in support for pytrees.  
    # This is useful - you can apply the SGD update using JAX pytree utilities.  
    return jax.tree.map(  
        lambda p, g: p - LEARNING_RATE * g, params, grads  
    )
```

Pytrees and JAX transformations

Many JAX functions (like `jax.lax.scan()`) operate on pytrees of array. In fact, all JAX transformations can be applied to functions that accept as input and produce as output pytrees of arrays.

Some JAX transformation take optional parameters that specify how certain input/output parameters should be treated. These parameters can also be pytrees, and their structure must correspond to the pytree structure of the corresponding arguments.

For example, we may have some arguments like this:

```
[ ]: args = (a1, {"k1": a2, "k2": a3}) # 3 leaf nodes (a1, a2, a3 - each are arrays)
```

```

# If we would like to apply vmap to a function that takes in a pytree
# of this shape, we must structure our in_axes the same way.
#
# In this example, we do not vmap over a1 or a2, but vmap over the columns of a3.
in_axes = (None, {"k1": None, "k2": 0})

jax.vmap(f, in_axes=in_axes)(args)

```

However, if we wanted to simply map over axis 0 for all leaf nodes, we can simply leave it as:

```

[ ]: jax.vmap(f, in_axes=0)(args)

```

5. Distributed Computing

5. Distributed Computing

```
[2]: import jax
import jax.numpy as jnp
```

SPMD is a parallelism technique where the same computation (e.g., a forward pass of a neural network) can be run on different input data (e.g., batch inputs) in parallel on different devices.

JAX has three SPMD strategies:

- Automatic sharding in JIT contexts.
- Explicit sharding which is similar to automatic sharding, but makes the sharding of an array part of the JAX type system as something queryable available at trace-time. The compiler still decides how to apply SPMD, but is constrained by user-supplied shardings.
- Fully manual sharding with per-device code and explicit communication collectives

```
[2]: # Tell JAX to create 8 logical CPUs, corresponding to 8 OS threads.
jax.config.update("jax_num_cpu_devices", 8)
print(jax.devices())
```

```
[CpuDevice(id=0), CpuDevice(id=1), CpuDevice(id=2), CpuDevice(id=3),
CpuDevice(id=4), CpuDevice(id=5), CpuDevice(id=6), CpuDevice(id=7)]
```

Data sharding

`jax.Array` represents an array in physical memory spanning one or more devices. The type is designed with distributed data and computation in mind.

Every `jax.Array` has a `jax.sharding.Sharding` object, which describes which shard of the global data is required by each device.

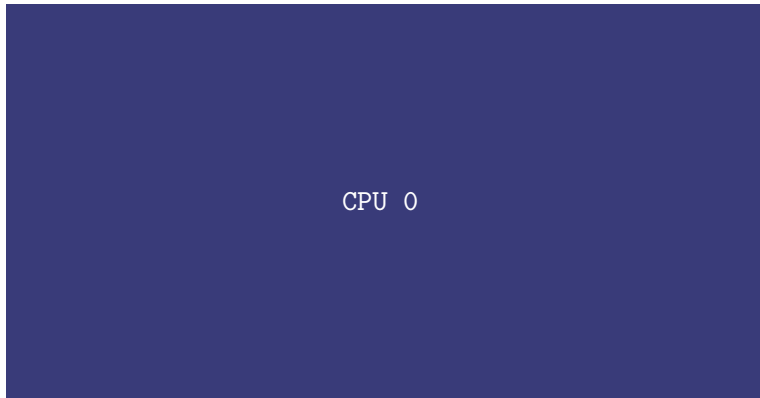
In simple cases, arrays are sharded on a single device:

```
[3]: a = jnp.arange(32.0).reshape(4, 8)
print(a.devices())
print(a.sharding)
```

```
{CpuDevice(id=0)}
SingleDeviceSharding(device=CpuDevice(id=0), memory_kind=device)
```

We can use `jax.debug.visualize_array_sharding()` to visualize how `a` is stored in memory of a single device:

```
[4]: jax.debug.visualize_array_sharding(a)
```



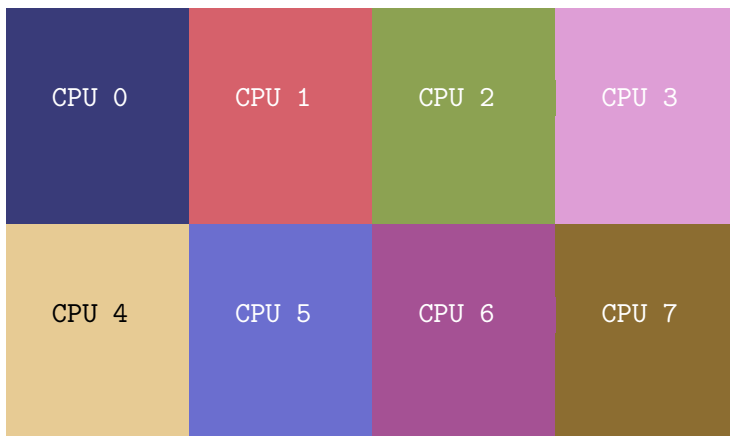
To create an array with non-trivial sharding, we can define an array with a `jax.sharding` specification and pass this to `jax.device_put()`:

```
[5]: # Make a (2, 4) grid of devices and label X-axis and Y-axis.
mesh = jax.make_mesh((2, 4), ("x", "y"))

# Create a specification to shard an array across the mesh.
# P("x", "y") means split the first dimension of an array across
# X-axis and second dimension across Y-axis.
sharding = jax.sharding.NamedSharding(mesh, jax.P("x", "y"))

# Shard the array across the mesh. It will be split into 2 chunks
# across the X-axis and 4 chunks across the Y-axis.
a = jnp.arange(32.0).reshape(4, 8)
a_sharded = jax.device_put(a, sharding)

jax.debug.visualize_array_sharding(a_sharded)
```



Automatic parallelism in JIT

Once we have shared data, the easiest way to do parallel computation is to simply pass the data to a JIT compiled function.

The XLA compiler includes heuristics for optimizing computations across multiple devices. We just need to specify how our input and output data is sharded.

```
[6]: @jax.jit
def f_elementwise(x):
    """Apply 2 * sin(x) + 1 to an array in element-wise fashion"""
    return 2 * jnp.sin(x) + 1

result = f_elementwise(a_sharded)
print("Shardings match:", a_sharded.sharding == result.sharding)
```

Shardings match: True

Another example is shown below:

```
[7]: @jax.jit
def f_contract(x):
    return jnp.sum(x, axis=0)

result = f_contract(a_sharded)
jax.debug.visualize_array_sharding(result)
```



The first two elements are replicated on devices 0 and 4, the second two elements on devices 1 and 5, and so on.

Explicit sharding

The key idea is that the JAX-level type of a value includes a description how the data is sharded.

The JAX-level type is effectively the information we have access to in a JIT function.

We can query the JAX-level type of any JAX value, Numpy array, or Python scalar with `jax.typeof()`:

```
[8]: a = jnp.arange(8)

# Use jax.typeof() to query the JAX-level type
print(f"JAX-level type:", jax.typeof(a))

@jax.jit
def f(a):
    # jax.typeof() also works in JIT contexts
    print(f"JAX-level type:", jax.typeof(a))
```

```
return 2 * a
```

```
f(a)
```

```
JAX-level type: int32[8]
```

```
JAX-level type: int32[8]
```

```
[8]: Array([ 0,  2,  4,  6,  8, 10, 12, 14], dtype=int32)
```

To see sharding in the type:

```
[9]: from jax.sharding import AxisType

mesh = jax.make_mesh((2, 4), ("X", "Y"), axis_types=(AxisType.Explicit,
↳AxisType.Explicit))

a = jnp.arange(8).reshape(4, 2)

# P("X", None) shards axis 0 across the X mesh (2 shards) and replicates each
↳shard
# across the Y mesh (4 replicas per shard). Y replicas hold identical data, so
↳any
# computation that doesn't differentiate across Y will be redundant across
↳those devices.
a_sharded = jax.device_put(a, jax.NamedSharding(mesh, jax.P("X", None)))

print(f"a type: {jax.typeof(a)}")
print(f"a_sharded type: {jax.typeof(a_sharded)}")
```

```
a type: int32[4,2]
```

```
a_sharded type: int32[4@X,2]
```

`int32[4@X,2]` means a 4 by 2 array of int32s, where the first dimension is sharded across the X-axis of the mesh. The array is replicated across the other mesh axes.

In this case, each device will receive a 2 by 2 array. The computation will be replicated across the Y-axis of the mesh.

This is an example of adding two sharded arrays:

```
[10]: # Explicitly shard along X-axis
a0 = jax.device_put(jnp.arange(4).reshape(4, 1), jax.NamedSharding(mesh, jax.
↳P("X", None)))

# Explicitly shard along Y-axis
a1 = jax.device_put(jnp.arange(8).reshape(1, 8), jax.NamedSharding(mesh, jax.
↳P(None, "Y")))
```

```

@jax.jit
def add(a, b):
    """Add two arrays. Will use Numpy style broadcasting."""
    ans = a + b
    print(f"a sharding: {jax.typeof(a)}")
    print(f"b sharding: {jax.typeof(b)}")
    print(f"ans sharding: {jax.typeof(ans)}")
    return ans

# Result will be sharded across both X and Y axes.
add(a0, a1)

```

```

a sharding: int32[4@X,1]
b sharding: int32[1,8@Y]
ans sharding: int32[4@X,8@Y]

```

```

[10]: Array([[ 0,  1,  2,  3,  4,  5,  6,  7],
             [ 1,  2,  3,  4,  5,  6,  7,  8],
             [ 2,  3,  4,  5,  6,  7,  8,  9],
             [ 3,  4,  5,  6,  7,  8,  9, 10]], dtype=int32)

```

Manual parallelism

In automatic parallelism, we write a function as if we are operation on the full dataset and JIT will split the computation across multiple devices.

In manual parallelism with `jax.shard_map()`, we write a function that will handle a single shard of data and `shard_map` will construct the full function:

```

[11]: mesh = jax.make_mesh((8,), ("X",))

def f_elementwise(x):
    """Apply 2 * sin(x) + 1 to an array in element-wise fashion"""
    return 2 * jnp.sin(x) + 1

a = jnp.arange(32)

# jax.shard_map() will take a per-shard function and create the full function.
f_elementwise_sharded = jax.shard_map(
    f_elementwise,
    mesh=mesh,
    in_specs=jax.P("X"), # Specify what axis to shard the input array along
    out_specs=jax.P("X"), # Specify what axis to shard the returned output
    ↪array along

```

```
)

af_sharded = f_elementwise_sharded(a)
print(af_sharded)
```

```
[ 1.          2.682942   2.818595   1.28224   -0.513605   -0.9178486
 0.44116902  2.3139732  2.9787164  1.824237  -0.08804226 -0.99998045
-0.07314587  1.840334   2.9812148  2.3005757  0.42419338 -0.92279494
-0.50197446  1.2997544  2.8258905  2.6733112  0.98229736 -0.69244087
-0.81115675  0.7352965  2.525117   2.912752   1.5418116  -0.32726777
-0.97606325  0.19192469]
```

For reference, `jax.shard_map()` can work inside a JIT function if needed.

The function only sees a single portion of the data:

```
[12]: x = jnp.arange(32)
print(f"Global shape: {x.shape}")

def f(x):
    print(f"Device-local shape: {x.shape}")
    return x * 2

y = jax.shard_map(f, mesh=mesh, in_specs=jax.P("X"), out_specs=jax.P("X"))(x)
```

```
Global shape: x.shape=(32,)
Device-local shape: x.shape=(4,)
```

Because the functions only see the device-local portion of the data, it means that aggregation-like functions require some extra thought.

For example, consider summation:

```
[13]: x = jnp.arange(32)

def f(x):
    # keepdims=True will return a 1D array instead of a scalar
    return jnp.sum(x, keepdims=True)

result = jax.shard_map(f, mesh=mesh, in_specs=jax.P("X"), out_specs=jax.
    P("X"))(x)
print(result)
```

```
[ 6  22  38  54  70  86 102 118]
```

Since `f` operates individually on each shard of the data, we never have any logic to sum across shards. We can explicitly handle this with collective operations like `jax.lax.psum()`:

```
[14]: x = jnp.arange(32)

def f(x):
    shard_sum = x.sum()

    # Performs an all-reduce style sum over all shards along X-axis
    return jax.lax.psum(shard_sum, "X")

# Note that out_specs=jax.P(); since the result is a scalar, we do not shard
# the result across the X-axis.
result = jax.shard_map(f, mesh=mesh, in_specs=jax.P("X"), out_specs=jax.P())(x)
print(result)
```

496

Comparing the three approaches

Consider a forward pass on a NN layer:

```
[15]: key = jax.random.key(1701)

@jax.jit
def layer(x, W, b):
    """Applies a dense nn layer with sigmoid activation."""
    return jax.nn.sigmoid(x @ W + b)

key, x_key, W_key, b_key = jax.random.split(key, 4)

x = jax.random.normal(x_key, (32,))
W = jax.random.normal(W_key, (32, 4))
b = jax.random.normal(b_key, (4,))
```

We can automatically perform this in distributed manner using `jax.jit()` and passing appropriately sharded data:

```
[16]: mesh = jax.make_mesh((8,), ("X",))
x_sharded = jax.device_put(x, jax.NamedSharding(mesh, jax.P("X")))
W_replicated = jax.device_put(W, jax.NamedSharding(mesh, jax.P())) # P() =
    ↪ replicate

# Every device will see 4 input elements. The weights are replicated across all
↪ devices.
# After each device completes its forward pass, the outputs will be aggregated
↪ together
# automatically, since JAX will insert a jax.lax.psum() call.
print(layer(x_sharded, W_replicated, b))
```

```
[0.01422223 0.9701333 0.00124826 0.08572862]
```

The weights are replicated across each device, and the input is sharded across the devices. The final result is joined together in all-reduce fashion.

We can also use explicit sharding:

```
[17]: explicit_mesh = jax.make_mesh((8,), ("X",), axis_types=(AxisType.Explicit,))
x_sharded = jax.device_put(x, jax.NamedSharding(explicit_mesh, jax.P("X")))
W_replicated = jax.device_put(W, jax.NamedSharding(explicit_mesh, jax.P()))

@jax.jit
def layer_auto(x, W, b):
    """Applies a dense nn layer with sigmoid activation."""
    print("x sharding:", jax.typeof(x))
    print("W sharding:", jax.typeof(W))
    print("b sharding:", jax.typeof(b))
    out = layer(x, W, b)
    print("out sharding:", jax.typeof(out))
    return out

# Every device will see 4 input elements. The weights are replicated across all
↳ devices.
# After each device completes its forward pass, the outputs will be aggregated
↳ together
# automatically, since JAX will insert a jax.lax.psum() call.
print(layer_auto(x_sharded, W_replicated, b))
```

```
x sharding: float32[32@X]
W sharding: float32[32,4]
b sharding: float32[4]
out sharding: float32[4]
[0.01422223 0.9701333 0.00124826 0.08572862]
```

The key difference here is that the sharding computation is still automatically handled by the compiler, but the sharding is now part of the JAX-level type system.

Finally, we can do the same thing with manual parallelism (i.e., `jax.shard_map()`):

```
[18]: from functools import partial

@jax.jit
@partial(
    jax.shard_map,
    mesh=mesh,
    in_specs=(
        jax.P("X"), # Shard `x` along X-axis
```

```

        jax.P("X", None), # Shard the rows of `W` along X-axis, columns are
↪replicated
        jax.P(None), # `b` is replicated across all devices
    ),
    out_specs=jax.P(None), # Output is replicated across all devices
)
def layer_sharded(x, W, b):
    # Each device computes its local partial (x_i @ W_i) in RM.
    # psum(..., "X") then does an elementwise all-reduce (sum) across devices,
    # yielding the full x @ W on every device. Adding b and sigmoid are local.
    return jax.nn.sigmoid(jax.lax.psum(x @ W, "X") + b)

print(layer_sharded(x, W, b))

```

```
[0.01422223 0.9701333 0.00124826 0.08572862]
```

Distributed Training

We can shard model weights across device and incrementally update them during training.

```

[3]: key = jax.random.key(1701)
mesh = jax.make_mesh((8,), ("X",))

B = 128 * jax.device_count()
D = 32
LEARNING_RATE = 0.01

# Generate dataset
X = jax.random.normal(key, (B, D))
w_true = jax.random.normal(key, (D, 1)) # True weights for label generation
y = (X @ w_true > 0).astype(jnp.float32) # Labels: 1 if positive, 0 else

# Shard data and parameters across X-axis
data_sharding = jax.NamedSharding(mesh, jax.P("X", None))
params_sharding = jax.NamedSharding(mesh, jax.P())

X_sharded = jax.device_put(X, data_sharding)
y_sharded = jax.device_put(y, data_sharding)

params = {"w": jnp.zeros((D, 1)), "b": jnp.zeros((1,))}
params_sharded = jax.tree.map(lambda a: jax.device_put(a, params_sharding),
↪params)

@partial(
    jax.jit,
    in_shardings=(params_sharding, data_sharding, data_sharding),

```

```

        out_shardings=params_sharding,
    )
def train_step(p, x, y):
    """
    Perform one step of training with data-parallel sharding.

    Args:
    - p: Model parameters replicated across all devices.
    - x: Input features of the batch sharded along the batch dimension (X-axis).
        In this example, each device receives 128 examples per step (128 * 8 / 8).
        Since there are 32 features, each device effectively has a 128x8
        matrix.
    - y: Target labels for the batch.
    """

    def loss_fn(p):
        logits = x @ p["w"] + p["b"]
        preds = jax.nn.sigmoid(logits)
        return jnp.mean((preds - y) ** 2)

    # Update weights & bias with loss gradient direction
    grad = jax.grad(loss_fn)(p)
    return jax.tree.map(lambda a, g: a - LEARNING_RATE * g, p, grad)

# Train for 100 steps
for _ in range(100):
    params = train_step(params_sharded, X_sharded, y_sharded)

```

```

-----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_30698/1261179473.py in <cell line: 0>()
      1 key = jax.random.key(1701)
----> 2 mesh = jax.make_mesh((8,), ("X",))
      3
      4 B = 128 * jax.device_count()
      5 D = 32

/usr/local/lib/python3.12/dist-packages/jax/_src/sharding_impls.py in
-> make_mesh(**kwargs)
    1178 axis_size = math.prod(new_axis_shapes)
    1179 if axis_size > len(devices):
-> 1180     raise ValueError(
    1181         f'Number of devices {len(devices)} must be >= the product '
    1182         f'of mesh_shape {new_axis_shapes}')

```

```
ValueError: Number of devices 1 must be >= the product of mesh_shape (8,)
```

The model's weights are replicated on every device, so each device holds a full copy and updates it in sync.

The input batch is sharded along the X-axis, so each device processes a different slice of examples, computes local gradients, and JAX automatically all-reduces those gradients so every device applies the same weight update.

6. Stateful computations

6. Stateful computations

JAX transformations like `jit()`, `vmap()`, `grad()`, require the functions they wrap to be pure: that is, functions whose outputs depend solely on the inputs, and which have no side effects such as updating of global state.

This constraint can pose some challenges in the context of machine learning, where state may exist in many forms. For example:

- Model parameters,
- Optimizer state, and
- Stateful layers.

Simple example: Counter

```
[1]: import jax
import jax.numpy as jnp

class Counter:
    """A simple counter."""

    def __init__(self):
        self.n = 0

    def count(self) -> int:
        """Increments the counter and returns the new value."""
        self.n += 1
        return self.n

    def reset(self):
        """Resets the counter to zero."""
        self.n = 0

counter = Counter()

for _ in range(3):
    print(counter.count())
```

2
3

`self.n` is the state that is maintained and modified as a side-effect during `count()` calls.

For the sake of example, assume we must JIT compile `count()` to improve performance.

```
[2]: counter.reset()
      fast_count = jax.jit(counter.count)

      for _ in range(3):
          print(fast_count())
```

1
1
1

Our counter isn't working. This is because the line `self.n += 1` in `count` involves a side effect: it modifies the input counter in-place, and so this function is not supported by jit. Such side effects are executed only once when the function is first traced, and subsequent calls will not repeat the side effect.

The solution: explicit state

Allow the user to manage the necessary state so that `count()` can be pure and therefore JIT compiled.

```
[3]: CounterState = int

class Counter:
    """A simple counter."""

    def count(self, n: CounterState) -> int:
        return n + 1

    def reset(self):
        return 0

c = Counter()
state = c.reset()

fast_count = jax.jit(c.count)

for _ in range(3):
    state = fast_count(state)
    print(state)
```

1
2

Linear Regression Example

Consider an example of a linear regression via gradient descent. Here, the only state is model parameters: The function to look at carefully is `update`.

```
[4]: from typing import NamedTuple

class Params(NamedTuple):
    weight: jnp.ndarray
    bias: jnp.ndarray

def init(rng) -> Params:
    """Returns the initial model params."""
    weights_key, bias_key = jax.random.split(rng)
    weight = jax.random.normal(weights_key, ())
    bias = jax.random.normal(bias_key, ())
    return Params(weight, bias)

def loss(params: Params, x: jnp.ndarray, y: jnp.ndarray) -> jnp.ndarray:
    """Computes the least squares error of the model's predictions on x against y.
    ↪ """
    pred = params.weight * x + params.bias
    return jnp.mean((pred - y) ** 2)

LEARNING_RATE = 0.005

@jax.jit
def update(params: Params, x: jnp.ndarray, y: jnp.ndarray) -> Params:
    """Performs one SGD update step on params using the given data."""
    grad = jax.grad(loss)(params, x, y)

    # If we were using Adam or another stateful optimizer,
    # we would also do something like
    #
    #     updates, new_optimizer_state = optimizer(grad, optimizer_state)
    #
    # and then use `updates` instead of `grad` to actually update the params.
    # (And we'd include `new_optimizer_state` in the output, naturally.)

    new_params = jax.tree.map(
        lambda param, g: param - g * LEARNING_RATE, params, grad)

    return new_params
```

Notice that we manually pipe the params in and out of the update function.

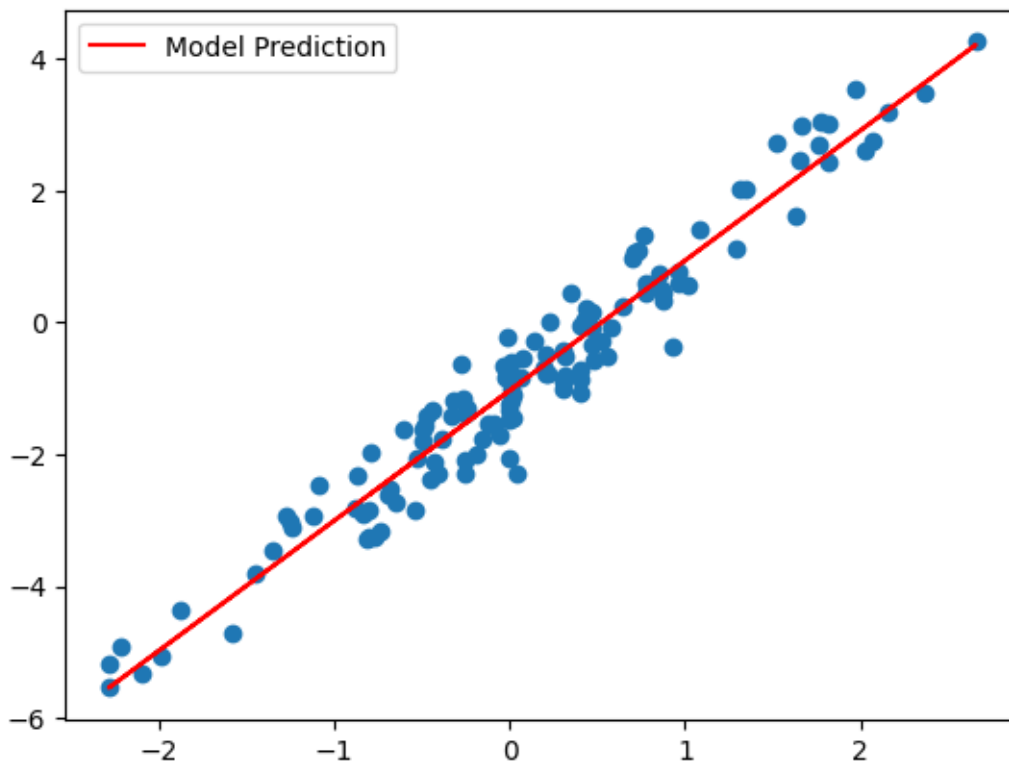
```
[5]: import matplotlib.pyplot as plt

rng = jax.random.key(42)

# Generate true data from  $y = w*x + b + \text{noise}$ 
true_w, true_b = 2, -1
x_rng, noise_rng = jax.random.split(rng)
xs = jax.random.normal(x_rng, (128, 1))
noise = jax.random.normal(noise_rng, (128, 1)) * 0.5
ys = xs * true_w + true_b + noise

# Fit regression
params = init(rng)
for _ in range(1000):
    params = update(params, xs, ys)

plt.scatter(xs, ys)
plt.plot(xs, params.weight * xs + params.bias, c='red', label='Model Prediction')
plt.legend();
```



```
[8]: from typing import NamedTuple

LEARNING_RATE = 0.005

class Params(NamedTuple):
    weights: jnp.ndarray
    biases: jnp.ndarray

def init(rng):
    """Initialize model weights and biases randomly"""
    W_key, b_key = jax.random.split(rng, 2)
    W = jax.random.normal(W_key, ())
    b = jax.random.normal(b_key, ())
    return Params(W, b)

def loss(params, x, y):
    """Compute the MSE of the model's prediction on x against y"""
    preds = params.weights * x + params.biases
    return jnp.mean((preds - y) ** 2)

@jax.jit
def update(params, x, y):
    loss_value, grads = jax.value_and_grad(loss)(params, x, y)

    # Adjust weights by gradient direction amplified by learning rate
    return loss_value, jax.tree.map(lambda param, g: param - g * LEARNING_RATE,
    ↪params, grads)
```

```
[9]: import matplotlib.pyplot as plt

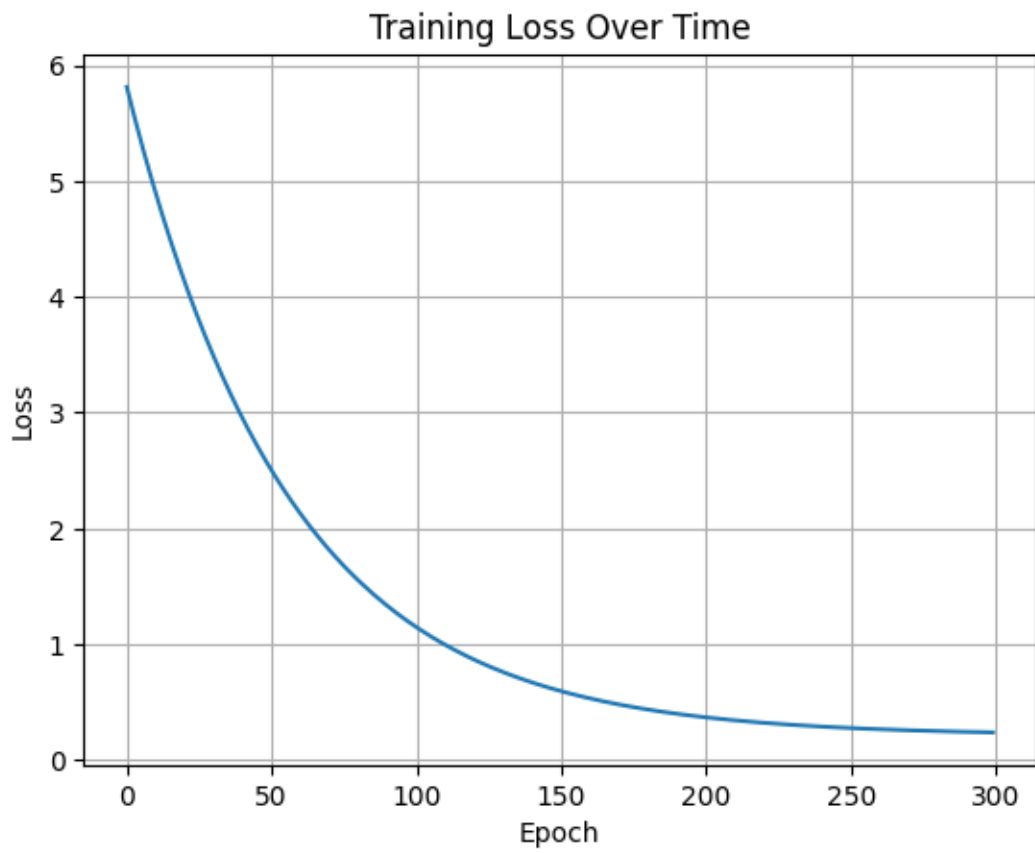
rng = jax.random.key(42)

# Generate true data from  $y = w*x + b + \text{noise}$ 
true_w, true_b = 2, -1
x_rng, noise_rng = jax.random.split(rng)
xs = jax.random.normal(x_rng, (128, 1))
noise = jax.random.normal(noise_rng, (128, 1)) * 0.5
ys = xs * true_w + true_b + noise

# Fit regression
params = init(rng)
losses = []
for i in range(300):
```

```
loss_value, params = update(params, xs, ys)
losses.append(loss_value)

plt.plot(losses)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Loss Over Time")
plt.grid(True)
plt.show()
```



7. Control flow and logical operators with JIT

7. Control flow and logical operators with JIT

When executing eagerly (outside of jit), JAX code works with Python control flow and logical operators just like Numpy code. Using control flow and logical operators with jit is more complicated.

In a nutshell, Python control flow and logical operators are evaluated at JIT compile time, such that the compiled function represents a single path through the control flow graph (logical operators affect the path via short-circuiting).

Paths dependent on the dtype or shape are okay, but will require recompilation when a value of a new dtype or shape is provided.

```
[1]: import jax
import jax.numpy as jnp
```

For example, this works:

```
[3]: from jax import jit
@jit
def f(x):
    for i in range(3):
        x = 2 * x
    return x

print(f(3))
```

24

So does this:

```
[4]: @jit
def g(x):
    y = 0.
    for i in range(x.shape[0]):
        y = y + x[i]
    return y

print(g(jnp.array([1., 2., 3.])))
```

6.0

But this doesn't, at least by default:

```
[5]: @jit
def f(x):
    if x < 3:
        return 3. * x ** 2
    else:
        return -4 * x

# This will fail!
f(2)
```

```
-----
TracerBoolConversionError                                Traceback (most recent call last)
/tmp/ipykernel_8711/3402096563.py in <cell line: 0>()
      7
      8 # This will fail!
----> 9 f(2)

[... skipping hidden 13 frame]

/tmp/ipykernel_8711/3402096563.py in f(x)
      1 @jit
      2 def f(x):
----> 3     if x < 3:
      4         return 3. * x ** 2
      5     else:

[... skipping hidden 1 frame]

/usr/local/lib/python3.12/dist-packages/jax/_src/core.py in error(self, arg)
    1804     if fun is bool:
    1805         def error(self, arg):
-> 1806             raise TracerBoolConversionError(arg)
    1807     elif fun in (hex, oct, operator.index):
    1808         def error(self, arg):

TracerBoolConversionError: Attempted boolean conversion of traced array with
↳shape bool[].
The error occurred while tracing the function f at /tmp/ipykernel_8711/
↳3402096563.py:1 for jit. This concrete value was not available in Python
↳because it depends on the value of the argument x.
See https://docs.jax.dev/en/latest/errors.html#jax.errors.
↳TracerBoolConversionError
```

Neither does this:

```
[6]: @jit
def g(x):
    return (x > 0) and (x < 3)

# This will fail!
g(2)
```

```
-----
TracerBoolConversionError                                Traceback (most recent call last)
/tmp/ipykernel_8711/543860509.py in <cell line: 0>()
      4
      5 # This will fail!
----> 6 g(2)

[... skipping hidden 13 frame]

/tmp/ipykernel_8711/543860509.py in g(x)
      1 @jit
      2 def g(x):
----> 3     return (x > 0) and (x < 3)
      4
      5 # This will fail!

[... skipping hidden 1 frame]

/usr/local/lib/python3.12/dist-packages/jax/_src/core.py in error(self, arg)
    1804     if fun is bool:
    1805         def error(self, arg):
-> 1806             raise TracerBoolConversionError(arg)
    1807     elif fun in (hex, oct, operator.index):
    1808         def error(self, arg):

TracerBoolConversionError: Attempted boolean conversion of traced array with
↳shape bool[].
The error occurred while tracing the function g at /tmp/ipykernel_8711/543860509.py:1 for jit. This concrete value was not available in Python because it
↳depends on the value of the argument x.
See https://docs.jax.dev/en/latest/errors.html#jax.errors.
↳TracerBoolConversionError
```

When we jit-compile a function, we usually want to compile a version of the function that works for many different argument values, so that we can cache and reuse the compiled code. That way we don't have to re-compile on each function evaluation.

The good news is that you can control this tradeoff yourself. By having jit trace on more refined abstract values, you can relax the traceability constraints. For example, using the `static_argnames` (or `static_argnums`) argument to jit, we can specify to trace on concrete values of some argu-

ments. Here's that example function again:

```
[7]: def f(x):  
    if x < 3:  
        return 3. * x ** 2  
    else:  
        return -4 * x  
  
f = jit(f, static_argnames='x')  
  
print(f(2.))
```

12.0

An example with a loop;

```
[8]: def f(x, n):  
    y = 0.  
    for i in range(n):  
        y = y + x[i]  
    return y  
  
f = jit(f, static_argnames='n')  
  
f(jnp.array([2., 3., 4.]), 2)
```

[8]: Array(5., dtype=float32)

Structured control flow primitives

There are more options for control flow in JAX. Say you want to avoid re-compilations but still want to use control flow that's traceable, and that avoids un-rolling large loops. Then you can use these 4 structured control flow primitives:

- `lax.cond` differentiable
- `lax.while_loop` fwd-mode-differentiable
- `lax.fori_loop` fwd-mode-differentiable in general; fwd and rev-mode differentiable if endpoints are static.
- `lax.scan` differentiable

Conditions

```
[9]: from jax import lax  
  
# Python version  
def cond(pred, true_fun, false_fun, operand):  
    if pred:  
        return true_fun(operand)  
    else:  
        return false_fun(operand)
```

```

# JAX Version
operand = jnp.array([0.])
lax.cond(True, lambda x: x+1, lambda x: x-1, operand)
# --> array([1.], dtype=float32)
lax.cond(False, lambda x: x+1, lambda x: x-1, operand)
# --> array([-1.], dtype=float32)

```

[9]: Array([-1.], dtype=float32)

LAX also provides other functions for conditions:

```

[10]: # 1) Choose between two precomputed values of the same shape and dtype
# lax.select(pred, a, b)
a = jnp.arange(10) * 10
b = jnp.arange(10) * 5
lax.select(a[-1] > b[-1], a, b)

# 2) Like lax.cond, but pick between any number of branch functions
# lax.switch(index, branches, operand)
branches = [lambda x: x + 1, lambda x: x - 1, lambda x: x * 2]
lax.switch(0, branches, 10)

```

[10]: Array(11, dtype=int32, weak_type=True)

while_loop

```

[11]: # Python version
def while_loop(cond_fun, body_fun, init_val):
    val = init_val
    while cond_fun(val):
        val = body_fun(val)
    return val

# JAX Version
init_val = 0
cond_fun = lambda x: x < 10
body_fun = lambda x: x+1
lax.while_loop(cond_fun, body_fun, init_val)
# --> array(10, dtype=int32)

```

[11]: Array(10, dtype=int32, weak_type=True)

fori_loop

```

[12]: # Python version
def fori_loop(start, stop, body_fun, init_val):
    val = init_val

```

```

    for i in range(start, stop):
        val = body_fun(i, val)
    return val

# JAX Version
init_val = 0
start = 0
stop = 10
body_fun = lambda i,x: x+i
lax.fori_loop(start, stop, body_fun, init_val)
# --> array(45, dtype=int32)

```

[12]: Array(45, dtype=int32, weak_type=True)

Logical operators

jax.numpy provides `logical_and`, `logical_or`, and `logical_not`, which operate element-wise on arrays and can be evaluated under jit without recompiling. Like their Numpy counterparts, the binary operators do not short circuit. Bitwise operators (`&`, `|`, `~`) can also be used with jit.

For example, consider a function that checks if its input is a positive even integer. The pure Python and JAX versions give the same answer when the input is scalar.

```

[13]: def python_check_positive_even(x):
    is_even = x % 2 == 0
    # `and` short-circuits, so when `is_even` is `False`, `x > 0` is not
    ↪evaluated.
    return is_even and (x > 0)

@jit
def jax_check_positive_even(x):
    is_even = x % 2 == 0
    # `logical_and` does not short circuit, so `x > 0` is always evaluated.
    return jnp.logical_and(is_even, x > 0)

print(python_check_positive_even(24))
print(jax_check_positive_even(24))

```

True

True

JAX version with `logical_and` returns elementwise values when applied to an array.

```

[14]: x = jnp.array([-1, 2, 5])
print(jax_check_positive_even(x))

```

[False True False]

Python logical operators error when applied to JAX arrays of more than one element, even without jit. This replicates NumPy's behavior.

```
[15]: print(python_check_positive_even(x))
```

```
-----  
ValueError                                Traceback (most recent call last)  
/tmp/ipykernel_8711/376774418.py in <cell line: 0>()  
----> 1 print(python_check_positive_even(x))  
  
/tmp/ipykernel_8711/3199076673.py in python_check_positive_even(x)  
      2 is_even = x % 2 == 0  
      3 # `and` short-circuits, so when `is_even` is `False`, `x > 0` is not_  
      ↪evaluated.  
----> 4 return is_even and (x > 0)  
      5  
      6 @jit  
  
/usr/local/lib/python3.12/dist-packages/jax/_src/array.py in __bool__(self)  
    307  
    308 def __bool__(self):  
--> 309     core.check_bool_conversion(self)  
    310     return bool(self._value)  
    311  
  
/usr/local/lib/python3.12/dist-packages/jax/_src/core.py in_  
    ↪check_bool_conversion(arr)  
    882             " `array.size > 0` to check that an array is not_  
    ↪empty." )  
    883     if arr.size > 1:  
--> 884         raise ValueError("The truth value of an array with more than one_  
    ↪element"  
    885             " is ambiguous. Use a.any() or a.all()")  
    886  
  
ValueError: The truth value of an array with more than one element is ambiguous _  
    ↪Use a.any() or a.all()
```

If you just want to apply grad to your python functions, without jit, you can use regular Python control-flow constructs with no problems, as if you were using Autograd (or Pytorch or TF Eager)

```
[17]: from jax import grad  
def f(x):  
    if x < 3:  
        return 3. * x ** 2  
    else:  
        return -4 * x  
  
print(grad(f)(2.)) # ok!
```

```
print(grad(f)(4.))  # ok!
```

12.0

-4.0